ASSIGNMENT 2 – CO2

CO2 Coverage: Design C++ programs by applying functions and object-oriented concepts – function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, specifying a class, member functions, access specifiers, arrays within a class, static data members and member functions, arrays of objects and friend functions.

NAME: P DILLI BABU

REG NO:192472284

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: 12-11-2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level) Mandatory Execution and GitHub Submission

GitHub Repository Link: https://github.com/dillibabu1122/DSA0115--C-/tree/main/AT2

Q1 – Call by Value, by Reference and by Pointer (6 Marks)

1.1 Three parameter-passing methods

Method	Parameter form	Effect on caller
Call by value	void f(long long fee)	A separate local copy is changed; caller's fee is unchanged.
Call by reference	void f(long long& fee)	The parameter aliases the caller's variable; changes affect caller.
Call by pointer	void f(long long* fee)	The pointer refers to caller's variable; *fee changes caller's value.

1.2 Prediction for the supplied program

3 12 13

100 13

16 18 16

Explanation: Initially a=1, b=2 and c=3. change(a,b,&c) adds 10 to its local copy a, to caller variable b through reference, and to caller variable c through pointer. Therefore a remains 1, b becomes 12 and c becomes 13. bigger(a,c) returns a reference to the larger variable. Since c=13 is larger than a=1, the assignment bigger(a,c)=100 changes c to 100. The second output should therefore be 1 100, not 100 13.

Correct exact output:

1 12 13

1 100

16 18 16

The show() calls use defaults from the right. show(1) uses y=5 and z=10, so prints 16. show(1,2) uses z=10, so prints 13. show(1,2,3) prints 6. Thus the final line is 16 13 6.

1.3 Corrected explanation and reference-return danger

The function bigger(int& p, int& q) returns a reference to one of its arguments. Both arguments are lvalues that remain alive after the function returns, so the returned reference is valid as long as the referenced variables remain alive.

1.4 Complete the code – caller's fee changed

```
void applyConcession(long long& fee, int percent)
{
   fee -= fee * percent / 100;
}
```

Expected output:

after call by value : 45000

after call by reference: 40500

Call by value changes only a local copy of 45000. The caller's hostelFee remains 45000. Call by reference aliases hostelFee, so subtracting 10% changes it to 40500.

1.5 Value, reference and pointer design decision

```
long long applyConcessionReturn(long long fee, int percent)
{
   return fee - fee * percent / 100;
}

// Usage:
// hostelFee = applyConcessionReturn(hostelFee, 10);
```

For several concessions, returning the updated fee is often easiest to test and compose: fee = applyConcessionReturn(fee, 10); fee = applyConcessionReturn(fee, 15);. In-place reference modification is suitable when the function's purpose is explicitly to update an existing fee. A pointer is useful when a null argument is meaningful, but it requires a null check.

All three approaches take O(1) time for one fee. Passing a long long by value copies one scalar (typically 8 bytes); reference and pointer pass an address (commonly 8 bytes on a 64-bit system). Passing a large Student object by value copies the whole object, whereas const reference avoids that copy and prevents modification. Use const Student& for read-only large objects.

Q2 – Return by Reference and Its Dangers (4 Marks)

```
int& highestMark(int marks[], int n)
{
   int best = 0;
   for (int i = 1; i < n; ++i)
      if (marks[i] > marks[best]) best = i;
   return marks[best];
}

// Correct use:
// int marks[] = {78, 91, 85};
// highestMark(marks, 3) += 2;
```

Returning int& allows the caller to modify the original array element, including assignment on the left side of '='. Returning const int& allows reading without allowing modification through that reference. Returning int returns a copy; changing the copy does not change the array.

```
int& dangerous()
{
   int local = 10;
   return local; // WRONG: returns reference to destroyed local object
}
```

A compiler commonly warns that a reference to a local variable is returned. After the function exits, local is destroyed; the reference dangles and using it has undefined behaviour. Do not suppress this warning.

Vector growth condition

A std::vector may reallocate its storage when it grows beyond capacity. Such reallocation invalidates references, pointers and iterators to its elements. A reference obtained before a push_back that triggers reallocation must not be used afterwards.

Index and pointer alternatives

```
int highestIndex(const std::vector<int>& marks) {
   int best = 0;
   for (int i = 1; i < static_cast<int>(marks.size()); ++i)
      if (marks[i] > marks[best]) best = i;
   return best;
```

```
}
// marks[highestIndex(marks)] += 2;

// Pointer alternative for a fixed array:
int* highestPointer(int marks[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[best]) best = i;
    return &marks[best];
}
```

Finding the maximum is O(n) time and O(1) extra space for reference, index and pointer versions. An index is often safest with a vector because it can be used again after reallocation (provided the logical element ordering has not changed and the index remains valid). A pointer or reference can become invalid after reallocation. Always check that n is positive before accessing element zero.

Q3 – Default Arguments and Function Overloading (6 Marks)

3.1 Why the original declarations are ambiguous

```
double charge(double weight);
double charge(double weight, bool express);
double charge(double weight, bool express = false,
              bool insured = false);
```

The one-argument call charge(2.0) can match the first overload and the third overload (using its two default arguments). The two-argument call can match the second overload and the third overload (using its final default argument). The compiler cannot choose uniquely, so the call is ambiguous.

3.2 Unambiguous redesigned overload set

```
#include <iostream>
struct Parcel {
    double weightKg;
    bool express;
    bool insured;
};
```

```
double charge(double weightKg, bool express = false,
              bool insured = false) {
    double amount = weightKg * 50.0;
    if (express) amount += 30.0;
    if (insured) amount += 20.0;
    return amount;
}
double charge(int weightGrams) {
    return charge(weightGrams / 1000.0, false, false);
}
double charge(const Parcel& p) {
    return charge(p.weightKg, p.express, p.insured);
}

int main() {
    Parcel p{2.0, true, true};
    std::cout << charge(2) << '\n';       // int overload
    std::cout << charge(2.0) << '\n';     // double overload
    std::cout << charge(2.0f) << '\n';    // float promotes to double
    std::cout << charge(2000) << '\n';    // int grams overload
    std::cout << charge(2.0, true) << '\n';
    std::cout << charge(p) << '\n';       // Parcel overload
}
```

Illustrative pricing policy used because the question does not provide actual courier rates: Rs 50 per kg, express surcharge Rs 30 and insurance Rs 20. These values demonstrate overload selection and should be replaced if the faculty has specified a tariff.

Call	**Selected overload**	**Reason**
charge(2)	charge(int)	Exact match to int.
charge(2.0)	charge(double, bool=false, bool=false)	Exact match to double; defaults fill remaining parameters.
charge(2.0f)	double overload	float-to-double is a promotion;

		no competing float overload.
charge(2000)	charge(int)	Exact match to int grams.
charge(2.0, true)	double overload	Exact matches for double and bool; insured uses default.
charge(p)	charge(const Parcel&)	Exact match to Parcel reference.

Using the illustrative rates, outputs are 100, 100, 100, 100, 130 and 150 respectively.

3.3 Overload resolution and defaults

- The compiler prefers an exact match over a promotion, and a promotion over a standard conversion, when ranking viable overloads.
- Default arguments are supplied from the right because trailing parameters may be omitted; a required parameter cannot follow a parameter with a default value.
- Default arguments do not create a separate overload. They make a declared function callable with fewer arguments.
- Overload resolution occurs at compile time and has no separate runtime cost.

3.4 Parcel structure or template alternative

A single Parcel structure centralizes optional properties and is easier to extend when adding fragile handling, destination or delivery windows. A set of overloads provides convenient syntax for distinct input types. A function template may reduce repeated code but needs constraints to avoid unintended matches. Each call remains O(1) for the simple calculation; passing Parcel by const reference avoids copying its fields.

Q4 – Specifying a Class with an Array Inside It (6 Marks)

```
#include <iostream>
#include <iomanip>

class SavingsAccount {
private:
   long long accountNumber;
   long long balancePaise;
```

```
    long long transactions[10];
    int count;

    void addTransaction(long long amount) {
        if (count < 10) {
            transactions[count++] = amount;
        } else {
            for (int i = 1; i < 10; ++i)
                transactions[i - 1] = transactions[i];
            transactions[9] = amount;
        }
    }

public:
    SavingsAccount(long long number, long long openingBalance)
        : accountNumber(number), balancePaise(openingBalance), count(0) {}

    void deposit(long long amountPaise);
    bool withdraw(long long amountPaise);
    void miniStatement() const;
};

void SavingsAccount::deposit(long long amountPaise) {
    if (amountPaise <= 0) return;
    balancePaise += amountPaise;
    addTransaction(amountPaise);
}

bool SavingsAccount::withdraw(long long amountPaise) {
    if (amountPaise <= 0 || amountPaise > balancePaise) return false;
    balancePaise -= amountPaise;
    addTransaction(-amountPaise);
    return true;
```

```
}

void SavingsAccount::miniStatement() const {
    std::cout << "Account: " << accountNumber
              << " Balance: Rs " << balancePaise / 100
              << '.' << std::setw(2) << std::setfill('0')
              << balancePaise % 100 << std::setfill(' ') << '\n';
    for (int i = 0; i < count; ++i)
        std::cout << (transactions[i] >= 0 ? "Deposit " : "Withdraw ")
                  << transactions[i] << " paise\n";
}

int main() {
    SavingsAccount account(10001, 500000);
    account.deposit(25000);
    account.withdraw(10000);
    account.miniStatement();
}
```

The array is private so outside code cannot corrupt transaction history or count. The member functions maintain the account invariant. The example stores deposits as positive paise and withdrawals as negative paise.

Shifting versus circular index

Method	Insertion when full	Cost
Shift array	Move entries 1–9 to positions 0–8, then write new entry at 9.	O(k), here O(10); general O(k).
Circular buffer	Overwrite oldest entry at the current write position and advance modulo capacity.	O(1) per insertion; O(k) storage.

For 50 transactions, change the fixed capacity from 10 to 50 and update the insertion logic. A circular buffer needs only a capacity constant and start/next/count indices, so increasing capacity changes storage

size but not the algorithm. std::deque can also use push_back and pop_front with O(1) operations at the ends and O(k) storage.

Circular-buffer implementation idea

```
// Fixed-capacity circular history fields:
long long transactions[50];
int start = 0;
int count = 0;

// Add:
if (count < 50) {
    transactions[(start + count) % 50] = amount;
    ++count;
} else {
    transactions[start] = amount;
    start = (start + 1) % 50;
}
```

Q5 – Static Members and Arrays of Objects (7 Marks)

5.1 Static member principles

- issued must be static because the number of seats issued belongs to the BusTicket class as a whole, not to one ticket.
- seat and name are non-static because each ticket has its own seat number and passenger name.
- A static member function has no this pointer and cannot directly access a non-static member; it can access static members.
- A static data member is declared inside the class and, before C++17 inline static use, defined once outside the class.
- An array of objects constructs each element; each constructor invocation can update the shared static count.

5.2 Prediction for supplied Token program

101 103 104 4

Token::next starts at 100. Constructing counter[3] calls the constructor three times and assigns IDs 101, 102 and 103. Constructing extra calls it once more and assigns 104. issued() returns next - 100 = 4.

5.3 Complete the BusTicket code

```
#include <iostream>
#include <string>

class BusTicket {
   static int issued;
   static const int CAPACITY = 3;
   int seat;
   std::string name;

public:
   explicit BusTicket(const std::string& n) : seat(0), name(n) {
      if (issued < CAPACITY) {
         ++issued;
         seat = issued;
      }
   }

   static int count() {
      return issued;
   }

   void print() const {
      std::cout << name << ": "
            << (seat ? "Seat " + std::to_string(seat)
                  : std::string("Waiting")) << '\n';
   }
};

int BusTicket::issued = 0;
```

```
int main() {
    BusTicket trip[4] = {
        BusTicket("Asha"), BusTicket("Bala"),
        BusTicket("Chitra"), BusTicket("Dev")
    };
    for (const BusTicket& t : trip) t.print();
    std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

Expected output:

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

5.4 Function called twice

```
void createTickets() {
    BusTicket a("Guest A");
    BusTicket b("Guest B");
}
// Calling createTickets() twice does not reset issued.
// The static count remains shared and continues increasing,
// until the capacity is reached.
```

Local ticket objects are destroyed when the function returns, but the static issued variable remains alive for the entire program. Therefore, calling the function twice does not reset the count. For independent buses, use one BusTicketOffice object per bus or a bus identifier with separate counts.

5.5 Global variable or TicketOffice alternative

A global counter and a static counter both take O(1) time and O(1) storage, but the global variable is less encapsulated and can be modified by unrelated code. A separate TicketOffice object passed to each ticket constructor also uses one counter per office, supports two buses independently, and makes the dependency explicit. A static counter is suitable when the application truly has only one bus/seat pool.

Q6 – Friend Functions and Inline Functions (8 Marks)

```
#include <iostream>
#include <cstdlib>

class Time {
   int h, m;

public:
   Time(int hh, int mm) : h(hh), m(mm) {}

   friend int minutesBetween(const Time& a, const Time& b);
   friend bool earlier(const Time& a, const Time& b);
};

int minutesBetween(const Time& a, const Time& b) {
   int first = a.h * 60 + a.m;
   int second = b.h * 60 + b.m;
   return std::abs(first - second);
}

bool earlier(const Time& a, const Time& b) {
   if (a.h != b.h) return a.h < b.h;
   return a.m < b.m;
}

int main() {
   Time departure(9, 45), arrival(13, 10);
   std::cout << minutesBetween(departure, arrival) << ' '
             << std::boolalpha
             << earlier(arrival, departure) << '\n';
}
```

Expected output: 205 false. 09:45 is 585 minutes after midnight; 13:10 is 790 minutes after midnight; the absolute difference is 205 minutes. arrival is not earlier than departure.

6.1 Why friend functions?

- A friend function is not a member, but the class grants it access to private and protected members.
- It is called like a normal function: minutesBetween(a,b), not a.minutesBetween(b).
- It is useful when an operation treats two objects symmetrically, such as comparing two Time objects.
- A member function is called on one object and naturally treats that object as the left-hand operand.
- Making h and m public would weaken encapsulation and allow any code to create invalid values such as hour=99 or minute=90.

6.2 Correctness tests

Test	Expected result
Time(09,45), Time(13,10)	minutesBetween = 205; earlier(first,second) = true
Time(13,10), Time(09,45)	minutesBetween = 205; earlier(first,second) = false
Time(12,00), Time(12,00)	minutesBetween = 0; earlier = false
Time(11,59), Time(12,01)	minutesBetween = 2; earlier = true

6.3 Inline meaning and timing experiment

The inline keyword permits a function definition in multiple translation units under the ODR rules and suggests that the compiler may substitute the function body at the call site. It does not force the compiler to inline the function. Optimizers may inline functions without the keyword, or decline to inline a function marked inline.

```
// Compile timing experiment with two builds:
g++ -std=c++17 -O0 time_test.cpp -o time_O0
g++ -std=c++17 -O2 time_test.cpp -o time_O2
// Run each and record elapsed time for 10,000,000 calls.
```

Do not claim a measured time without running the benchmark on the target computer. Use the same input, compiler version and number of calls for both builds. At -O2 the compiler may inline and optimize the small functions, so the optimized version is often faster; at -O0 function-call overhead may be more visible. The exact result depends on compiler, CPU, measurement method and whether the benchmark prevents dead-code elimination.

6.4 Member/getter alternative and deployment decision

```
class TimeWithMembers {
   int h, m;
public:
   TimeWithMembers(int hh, int mm) : h(hh), m(mm) {}
   int minutesTo(const TimeWithMembers& other) const {
      int a = h * 60 + m;
      int b = other.h * 60 + other.m;
      return std::abs(a - b);
   }
   bool isEarlier(const TimeWithMembers& other) const {
      return h < other.h || (h == other.h && m < other.m);
   }
};
```

Member functions keep access within the class and avoid granting friendship. Getters also preserve private data but expose more interface methods. All versions perform O(1) work per comparison; after optimization, inlining can make their performance similar. Recommendation: keep the friend functions if the operations are genuinely symmetric and the class wants a minimal public interface; otherwise use const member functions for simpler ownership semantics.

Q7 – DAY-4 DESIGN CHALLENGE: Diagnose, Redesign and Validate (13 Marks)

Selected program: Q4 SavingsAccount transaction history

The original fixed array implementation shifts the remaining k−1 transactions whenever a new transaction arrives after the array is full. The cost of one full-buffer insertion is O(k). This is a clear design limitation as the required history grows from 10 to 50 or more transactions.

A. Diagnose the limitation (2 Marks)

When the history is full, adding one transaction requires moving every retained transaction one position toward the beginning. With capacity k, one insertion performs approximately k−1 assignments. The design is correct but performs unnecessary movement because the oldest transaction is the only item that must be discarded.

B. Define measurable objective (2 Marks)

Objective: preserve the newest k transactions in chronological order while reducing the work of adding a transaction from O(k) shifts to O(1) writes and index updates. Memory must remain O(k), and miniStatement() must print the retained transactions from oldest to newest.

C. Revised circular-buffer design (4 Marks)

```
#include <iostream>
#include <string>

class TransactionHistory {
private:
    static const int CAPACITY = 50;
    long long values[CAPACITY];
    int start; // index of oldest retained transaction
    int count;

public:
    TransactionHistory() : start(0), count(0) {}

    void add(long long amount) {
        if (count < CAPACITY) {
            values[(start + count) % CAPACITY] = amount;
            ++count;
        } else {
            values[start] = amount;       // overwrite oldest
            start = (start + 1) % CAPACITY;
        }
    }

    void print() const {
        for (int i = 0; i < count; ++i) {
            int index = (start + i) % CAPACITY;
            std::cout << values[index] << " ";
```

```
        }
        std::cout << '\n';
    }

    int size() const { return count; }
};

int main() {
    TransactionHistory history;
    for (int i = 1; i <= 55; ++i)
        history.add(i * 100);
    history.print();
    std::cout << "Stored: " << history.size() << '\n';
}
```

Expected output for the sample:

```
600 700 800 900 1000 ... 5400 5500
Stored: 50
```

The history contains the latest 50 values, from 600 through 5500. The abbreviated line above describes the sequence; actual program output prints every retained value.

Circular-buffer pseudocode / class relationship

```
TransactionHistory
  - values[50]
  - start
  - count
  + add(amount)
  + print()
  + size()

add(amount):
  IF count < CAPACITY:
      values[(start + count) MOD CAPACITY] = amount
      count = count + 1
```

```
ELSE:
    values[start] = amount
    start = (start + 1) MOD CAPACITY
END IF
```

Extra memory remains O(k), with a fixed array of 50 long long values and two integer indices. The redesign does not require dynamic allocation or a library container. It changes insertion logic, not the public account behaviour.

D. Predict before implementing (1 Mark)

Prediction: when the buffer is full, a new transaction will replace exactly the oldest transaction. Each insertion will require a fixed number of array/index operations rather than up to 49 element shifts. The retained history size will never exceed 50.

E. Implement and validate (2 Marks)

Test	Input sequence	Expected result
Below capacity	Add 1 to 10	Print 1 through 10; count 10
Exactly capacity	Add 1 to 50	Print 1 through 50; count 50
Above capacity	Add 1 to 55	Print 6 through 55; count 50
Repeated overflow	Add 1 to 100	Print 51 through 100; count 50

The sample C++ program implements the above-capacity case. Execute it and capture the actual output. To compare speed fairly, create a second benchmark using the original shifting algorithm and the circular buffer, feed both the same large sequence, compile with the same optimization level and report elapsed time. The actual measured timings must be filled in from your computer.

F. Defend the final design (2 Marks)

Measure	Shift array	Circular buffer
Insertion when full	O(k) assignments	O(1) overwrite and index update
Storage	O(k)	O(k)
Chronological printing	O(k)	O(k), with modular indexing
Implementation	Simpler to understand for very small k	Slightly more index logic

Best use	Tiny history where k is very small	Growing history or frequent insertions

The objective is achieved at the algorithmic level: full-buffer insertion changes from O(k) to O(1), while memory remains O(k). The original shifting method may still be preferable for a very small history because it is straightforward and its constant cost is tiny. For a 50-entry or larger history with frequent updates, the circular buffer is the better design.

Conclusion

The solutions demonstrate how C++ parameter passing, references, overloads, classes, static members, friend functions and inline functions can be applied to practical systems. The most important design choices are to avoid dangling references, prevent ambiguous overloads, encapsulate class data, use static state only when it truly belongs to the class, and select a circular buffer when frequent transaction-history updates make shifting inefficient.